

Network Performance Analysis under Impairment

A controlled study of TCP throughput under packet loss, limited bandwidth, and delay

Hajer Metoui · ESSAT, Computer Engineering (Networks & Telecommunications) · 2026

1. Introduction

Network connections rarely behave perfectly: real links have limited capacity, introduce delay, and occasionally lose data. A common everyday observation motivates this study — even a small amount of packet loss (a percent or two) can degrade an interactive application, such as an online game, far more severely than the small percentage would suggest. This report asks a simple question: *how do packet loss, available bandwidth, and delay each affect the throughput of a TCP connection?*

To answer it, a small network was emulated and its bottleneck link systematically impaired while throughput was measured. Each factor was varied independently so its effect could be isolated.

2. Background

TCP (Transmission Control Protocol) is the protocol most applications use for reliable data transfer. It guarantees delivery by retransmitting lost data and it controls its sending rate through *congestion control*: when it detects loss, it interprets this as a sign of network congestion and deliberately reduces its rate. This mechanism is central to interpreting the results below.

Throughput is the amount of data successfully transferred per second (here, in Mbit/s). **Mininet** is a network emulator that creates virtual hosts and switches on a single machine; **iperf3** measures throughput between two hosts; and Linux traffic control allows a link to be given a specific bandwidth, delay, and loss rate.

3. Methodology

The experimental network was built in Mininet with four hosts and two switches (`h1` , `h2` on switch `s1` ; `h3` , `h4` on switch `s2`). The two switches were joined by a single *bottleneck link* whose bandwidth, delay, and loss could be configured. Throughput was measured with iperf3 between `h1` and `h3` , so all traffic traversed the bottleneck.

Three experiments were run. In each, one parameter was varied while the other two were held fixed, so that any change in throughput could be attributed to the varied parameter alone:

Experiment	Varied	Held fixed
A — Packet loss	0, 1, 2, 5, 10 %	bw = 10 Mbit/s, delay = 5 ms
B — Bandwidth	1, 5, 10, 20, 50 Mbit/s	loss = 0, delay = 5 ms
C — Delay	1, 10, 50, 100, 200 ms	bw = 10 Mbit/s, loss = 0

A capped bandwidth (rather than an unlimited link) was used deliberately: an uncapped emulated link reflects host hardware speed rather than network behaviour, so a fixed cap gives a stable, realistic bottleneck against which the effects of loss and delay can be measured.

4. Results

Packet loss. Throughput fell sharply and non-linearly as loss increased. Low loss had little effect on throughput but caused a large rise in retransmissions; beyond a few percent, throughput collapsed.

Loss (%)	0	1	2	5	10
Throughput (Mbit/s)	9.41	9.19	7.11	5.23	1.67
Retransmissions	0	32	89	109	72

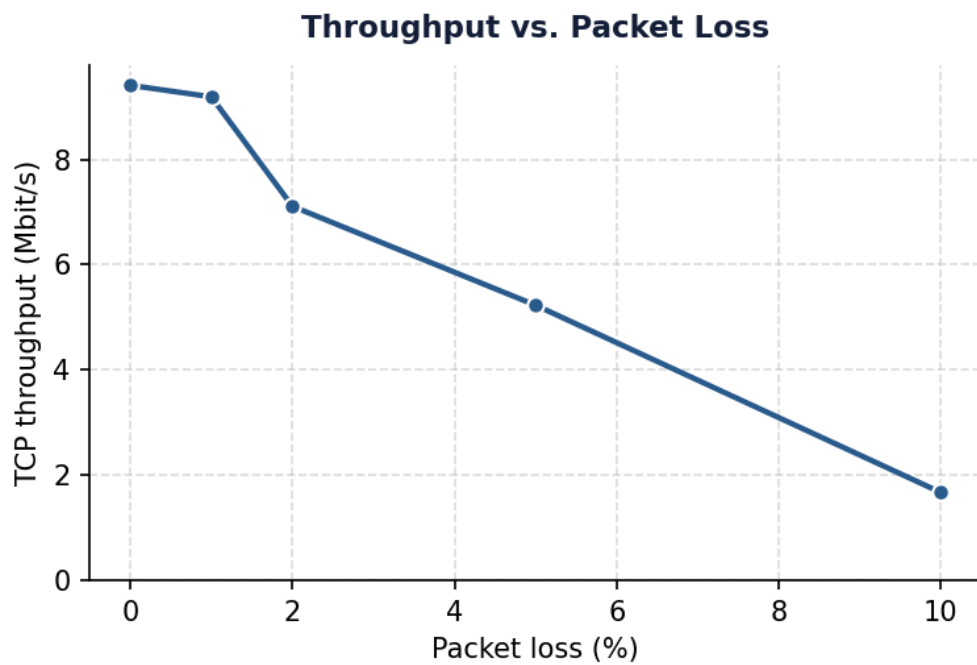


Figure 1. Throughput collapses non-linearly as packet loss rises.

Bandwidth. Throughput tracked the configured bandwidth almost exactly, scaling linearly across the tested range.

Bandwidth (Mbit/s)	1	5	10	20	50
Throughput (Mbit/s)	0.90	4.72	9.44	18.9	47.4

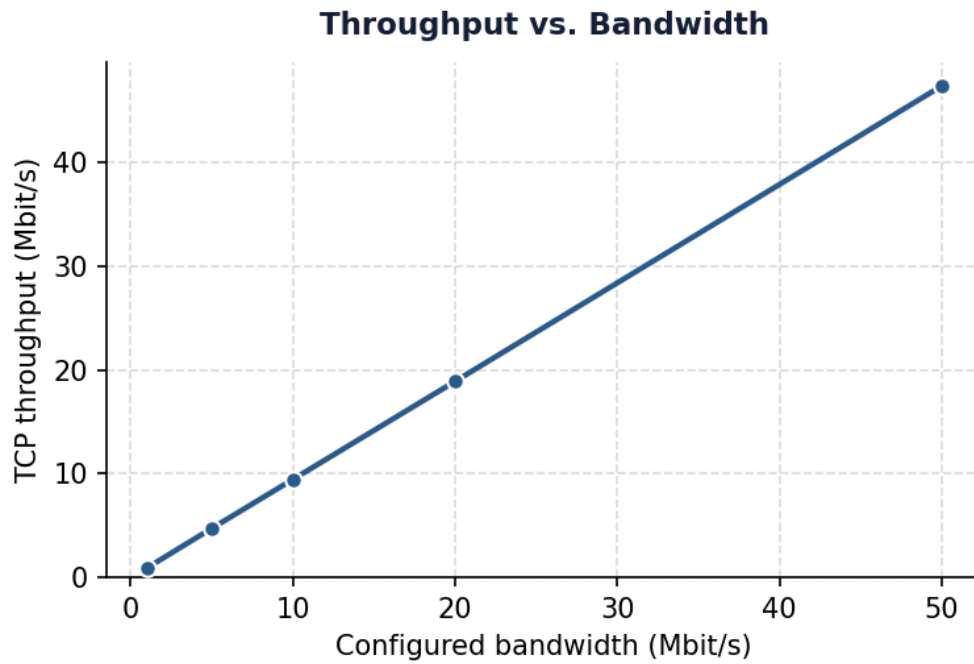


Figure 2. Throughput scales linearly with configured bandwidth.

Delay. Throughput was nearly unaffected by small delays but declined gradually as delay grew large.

Delay (ms)	1	10	50	100	200
Throughput (Mbit/s)	9.42	9.37	8.95	8.08	6.47

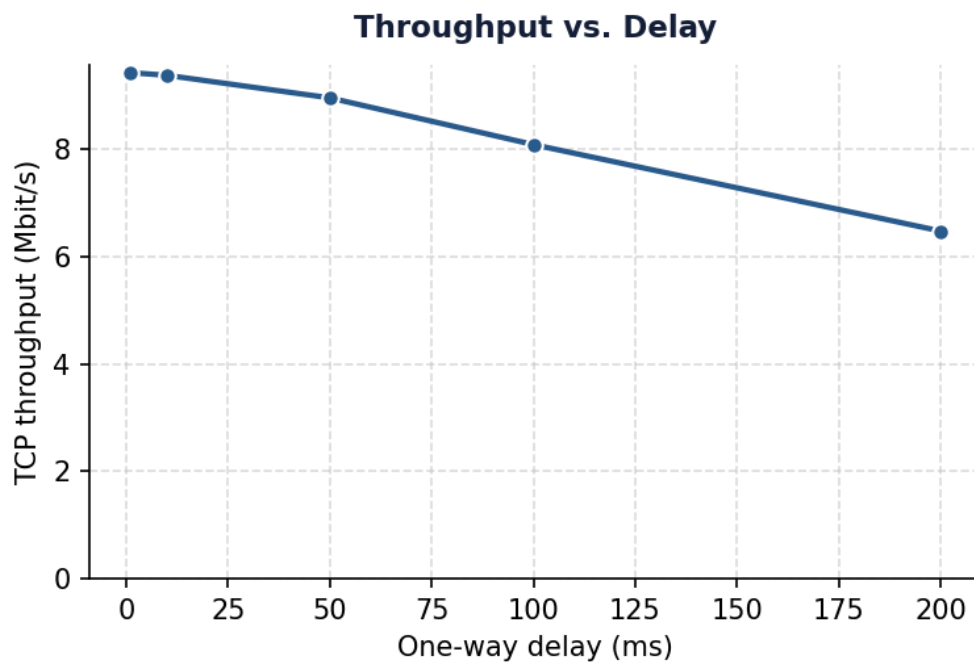


Figure 3. Throughput declines gradually as delay increases.

5. Discussion

The three factors produced three distinct behaviours. **Bandwidth** acts as a simple ceiling: throughput rises linearly with it, as expected, since the link capacity directly limits the transfer rate.

Packet loss is the most damaging and the least intuitive. At 1% loss, throughput barely changed (9.41 → 9.19 Mbit/s) yet retransmissions jumped from 0 to 32 — the connection was working much harder to hide the loss. As loss increased further, throughput collapsed (down to 1.67 Mbit/s at 10%). This is a direct consequence of TCP congestion control: TCP treats loss as congestion and sharply reduces its sending rate, so a small loss rate triggers a disproportionately large slowdown. This explains the everyday observation that motivated the study — why a seemingly small loss percentage can make an interactive application feel severely degraded. Notably, retransmissions *fell* at 10% loss (72, below the 109 seen at 5%): by that point TCP had throttled itself so heavily that it was sending too little data for many retransmissions to occur.

Delay degrades throughput gently and for a different reason. TCP can only keep a limited amount of unacknowledged data in flight before it must wait for confirmation; as the round-trip time grows, more time is spent waiting, leaving the link underused. Hence throughput declined gradually rather than collapsing.

6. Limitations

This study uses emulation on a single machine rather than physical hardware, so absolute figures reflect that environment. Each configuration was measured over a short interval, and the per-second throughput was variable — particularly at high delay, where TCP sent data in larger, burstier waves — so results indicate trends rather than precise values. Only one loss/bandwidth/delay dimension was varied at a time; interactions between them were not explored. These are appropriate simplifications for an introductory controlled study.

7. Conclusion

Across all three experiments, the impairments affected TCP throughput in clearly different ways: bandwidth scaled it linearly, delay reduced it gradually, and packet loss degraded it sharply and non-linearly. The loss result, in particular, gives a concrete, measured explanation for why small amounts of packet loss can cause disproportionately large performance degradation — a direct effect of TCP's congestion-control response to loss. Natural extensions would include repeating each configuration multiple times to quantify variability, and studying combinations of impairments together.